

Classes and objects in python

Introduction (recap of classes and objects) :

recall classes and objects in python. An class is just a data type, remember the famous line “everything in python is an object”, they’re not saying every variable, they say everything, even a function is some kind of object.

An **object** of any type/kind will have some **properties and functionalities**.

- In this case, by properties we mean like current values or some parameters that define the current state of that object. Like a chalk piece object can have properties like `current_length`, `original_full_length`, `material`, ... and many other properties like how much length we used up on the chalk piece’s north pole and south pole, and what’s the shape at both ends... THE MAIN IDEA IS THAT --> , the object properties should be detailed enough to describe the current state of the whole object.
(in the above chalk piece example, we could discard the `current_length` property if we know original length and how much we chewed on both ends)
- In python objects we use the term “data members” for properties. (in python objects we use simple objects, not like above example too many properties, like we store some variables, other objects....., don’t worry)
- And functionalities : every object has some functionalities, or features. These features do some display actions or just change any global variables or its own data members mainly.
- For example consider a car object, it has a simple data member (one of many) called `on_or_off`. This object could have functions like `start`, `stop`. `start` function just sets the `on_or_off` values to true and `stop` sets it to false, just changing own object’s data members
- In python objects we use the term “methods” for functionalities. Again through *methods* we can provide all functionalities this object can support

Class : a blue print for a particular type/class of objects. In terms of python, class just defines a datatype/objecttype. Objects are just instances of the class

- example : cars is just a category of motor vehicles, but every individual car is a different object. The class ‘car’ just defines how every ‘car’ object should be like, what data members it should contain and what methods it should support
- in python, we open a block of code called “class” with a class name, for example “X”, this block contains list of methods the objects of this class supports, and data members we write inside methods only. Example below

```
1 class buggati_chiron:      # its just a famous car
2
3     def start(self):
4         # start is a method of the class
5         # it just sets the data member on_or_off to true
6         self.on_or_off = True
7
8     def stop(self):
9         # stop is a method of the class
10        # it just sets the data member on_or_off to false
11        self.on_or_of = False
12
13    def refuel(self,x):
14        # refuel is a method of the class
15        # it just increments the fuel_level by x
16        self.fuel_level += x
```

ignore self for now, consider only the start, stop functions. We can create any data member by just, inside any method in the class, start a variable like `self.on_or_off` and initialize it to something. And that creates a data member called “on_or_off” in your object.

In this example refuel isn’t exactly creating a data member `fuel_level`, we are incrementing a `fuel_level` data member that doesn’t exist. We should’ve created it before. And do not run this above garbage, we’ll rewrite it better below

```

1  class buggati_chiron:      # its just a famous car
2
3      def start(self):
4          self.on_or_off = True
5
6      def stop(self):
7          self.on_or_off = False
8
9      def paint(self, paint_color):
10         self.color = paint_color
11
12
13  bc1 = buggati_chiron()
14  # object created, this object can be referenced using variable 'bc1'
15  # but no data members created for this object yet
16  # for creating/initializing data members for this call start or stop
17  # when they run, they run the lines self.on_or_off = True/False
18  # only then the particular data member is created
19
20  bc1.stop()
21  # you can invoke an object's methods this way directly
22  # object variable, dot, method/function name,
23  # and provide any arguments the function may take
24
25  print(bc1.on_or_off)  # we can access data members outside the class this way
26  bc1.start()
27  print(bc1.on_or_off)
28  bc1.stop()
29  print(bc1.on_or_off)
30
31  bc1.paint("red")
32  print(bc1.color)
33
34  # the class just contains methods/functions
35  # data members initialization happens when the function runs the line self.dat_mem = ....something
36  # from outside we can change the data members directly(only after they are created)
37  # and call methods (using the object's features basically)

```

first write the class above and then to create an object of this class , just do `bc1 = buggati_chiron()`. `bc1` is the variable we use to reference to the object created by `buggati_chiron()` piece. Note that this is not a function, just looks like a function call but its not, we'll see.

We wrote a start, stop function both of them creates the data member `on_or_off` if not yet created and sets true or false, and a paint function, which takes an argument called `paint_color`. And creates the data member called 'color' and sets it to `paint_color` given by the user, in this case red

we haven't seen constructors, `__init__` yet, we'll get there, wait. Another example

```

1  class Student:
2      def set_name(self, name):
3          self.name = name
4
5      def get_name(self):
6          # like any function, methods can return something too
7          # also methods can take arguments like normal functions
8          return self.name
9
10     def set_rollno(self, rollno):
11         self.roll = rollno
12
13  mike = Student()
14
15  mike.set_name("mike tyson")
16  something = mike.get_name()
17  print(something)
18  print(mike.name)
19  # both lines print same thing
20  # the get function returns the name, or we can return anything
21
22  mike.set_rollno(1)
23  print(mike.roll) # prints 1
24
25  # printing mike.roll before calling mike.set_roll(1) would give error
26  # like data member or attribute doesn't exist

```

The need for constructors :

The problem with above Student class is that, we should've done all the data members initialization at the start. Currently the user has to write very carefully, if he wants to use a method that uses a particular data member, he needs to first call some other function that are responsible for initializing the data members that he's trying to access

like for example if in the previous car example if we have a function like get_on_off_status which just returns on_or_off value, we should've first called start or stop functions coz they are the ones creating the data member before this get_on_off_status can access it.

So ideally, we use constructors for every class which is just a function that runs when a new object of the class is created. A constructor is just a function with name `__init__`. When we do `bc1 = buggati_chiron()` or `mike = Student()` the constructor method first runs (not in these examples coz we didn't write it yet). In the constructor function we can create/ initialize all the data members and eliminate the above problem

Actually the main job of a constructor in general is to initialize certain values of some data members, only in python we have this extra problem that we need to create them data members too, in c++ or java data members are declared first, in python they only get declared when any function initializes `self.data_member` only

in python constructor : initialize necessary data members and set initial values . Whenever an object is created, first of all this constructor function will run automatically for this particular object. This constructor function doesn't return anything but it can take any number of arguments. We write a constructor using 'def __init__(self,...):'

now lets see above student example with a constructor

this all writing is to explain the importance of a constructor, the highlighted part is the main takeaway

```
comp.py x
1 class Student:
2     def __init__(self,name,rollno):
3         self.Name = name
4         self.roll_no = rollno
5         self.GPA = 0
6         print("this is the constructor function. object created!")
7
8     def print_info(self):
9         print("the student's name is", self.Name,
10             "and his roll no is", self.roll_no)
11
12     def set_gpa(self,gpa):
13         self.GPA = gpa
14
15     def get_gpa(self):
16         return self.GPA
17
18 mike = Student("mike_tyson",20)
19 mike.print_info()
20 mike.set_gpa(9.5)
21 print(mike.get_gpa())
```

```
revanth@victus: ~/Desktop
~/Desktop
revanth@victus:~$ cd Desktop/
revanth@victus:~/Desktop$ python3 comp.py
this is the constructor function. object created!
the student's name is mike_tyson and his roll no is 20
9.5
revanth@victus:~/Desktop$
```

in this example mike is the object we created, here we wrote the constructor function (it is also a function, can take arguments, but no return statement is allowed)

this constructor takes 2 arguments name and rollno. We pass them using `mike = Student("mike_tyson",20)` line

then the constructor function runs, and the data members Name, roll_no, GPA are initialized and the print statement runs. Then the control goes to line 19 and continues. The class is just a blue print, there is nothing to run inside, from outside the class only we create objects and their method function's code exists in the class, so executing starts at line 18, then 19,...

in java and c++ the object oriented concepts are very different but the idea is same. A class is a blue print for a group of objects of same type. (a special datatype for objects basically) and we create individual objects of this type and manipulate the objects properties using the methods

here we will not talk about inheritance and other OOPS concepts like encapsulation, abstraction, polymorphism,....

Note : even a single int number in python is an object of int type, its not a simple 4byte variable like in c

When we write `a = 4` somewhere. The value 4 is an integer object with its own properties and methods, maybe very few properties, it just has to store the value 4, but methods are there. And 'a' is just a variable referencing to this object